

BACKEND & DATABASE SCHEMA

What's Under the Hood

SQLite schema, Rust module structure, Tauri command surface, and the Ollama data contract.

What's Inside

01 Database Overview

File layout, journaling mode, and entity relationships.

02 Schema Reference

Full DDL for every table, grouped by domain.

03 Rust Backend Structure

The `src-tauri` module layout behind the command layer.

04 Tauri Command Surface

Every command the frontend is allowed to invoke.

05 Ollama Contract (Rust Side)

The `serde` structs mirroring the frontend `Zod` schema.

06 Indexing & Performance

What makes single-item lookup sub-millisecond.

07 Migration Strategy

How the schema evolves without risking local data.

01 — STRUCTURE

Database Overview

All persistence lives in a single `aurafit.db` SQLite file inside the OS-standard `app-data` directory, accessed exclusively through `tauri-plugin-sql`. WAL (write-ahead log) mode is enabled for safe concurrent read/write between the UI thread and background writes.

Profile & Targets

`user_profile`, `goals`,
`dietary_guardrails`

Logging

`food_log`, `food_log_items`,
`weight_history`

Reference & Recipes

`usda_foods`, `recipes`,
`recipe_ingredients`,
`app_settings`

Relationship	Type
<code>food_log</code> → <code>food_log_items</code>	One-to-many (a logged meal has 1..n items)
<code>food_log_items</code> → <code>usda_foods</code>	Optional many-to-one (set only for Quick Lookup entries)
<code>recipes</code> → <code>recipe_ingredients</code>	One-to-many
<code>user_profile</code> → <code>goals</code>	One-to-many, exactly one <code>is_active = 1</code> at a time

02 — DDL

Schema Reference

2.1 Profile & Targets

```
CREATE TABLE user_profile (  
  id INTEGER PRIMARY KEY CHECK (id = 1),  
  sex TEXT NOT NULL CHECK (sex IN ('male','female')),  
  date_of_birth TEXT NOT NULL,  
  height_cm REAL NOT NULL,  
  activity_level TEXT NOT NULL CHECK (activity_level IN  
    ('sedentary','light','moderate','active','very_active')),  
  created_at TEXT NOT NULL DEFAULT (datetime('now')),  
  updated_at TEXT NOT NULL DEFAULT (datetime('now'))  
);  
  
CREATE TABLE goals (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  goal_type TEXT NOT NULL CHECK (goal_type IN  
    ('aggressive_fat_loss','lean_bulk','recomposition','maintenance')),  
  target_calories INTEGER NOT NULL,  
  target_protein_g REAL NOT NULL,  
  target_carbs_g REAL NOT NULL,  
  target_fat_g REAL NOT NULL,  
  target_fiber_g REAL NOT NULL,  
  is_active INTEGER NOT NULL DEFAULT 1,  
  created_at TEXT NOT NULL DEFAULT (datetime('now'))  
);  
  
CREATE TABLE dietary_guardrails (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  constraint_type TEXT NOT NULL CHECK (constraint_type IN ('diet','allergy')),  
  value TEXT NOT NULL,  
  is_active INTEGER NOT NULL DEFAULT 1,  
  UNIQUE(constraint_type, value)  
);
```

2.2 Logging

```
CREATE TABLE food_log (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  logged_at TEXT NOT NULL DEFAULT (datetime('now')),  
  source TEXT NOT NULL CHECK (source IN  
    ('vision_ai', 'quick_lookup', 'manual', 'recipe')),  
  photo_path TEXT,  
  total_calories REAL NOT NULL DEFAULT 0,  
  total_protein_g REAL NOT NULL DEFAULT 0,  
  total_carbs_g REAL NOT NULL DEFAULT 0,  
  total_fat_g REAL NOT NULL DEFAULT 0  
);  
  
CREATE TABLE food_log_items (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  food_log_id INTEGER NOT NULL REFERENCES food_log(id) ON DELETE CASCADE,  
  usda_fdc_id INTEGER REFERENCES usda_foods(fdc_id),  
  name TEXT NOT NULL,  
  estimated_grams REAL NOT NULL,  
  calories REAL NOT NULL,  
  protein_g REAL NOT NULL,  
  carbs_g REAL NOT NULL,  
  fat_g REAL NOT NULL,  
  confidence TEXT CHECK (confidence IN ('low', 'medium', 'high')),  
  user_edited INTEGER NOT NULL DEFAULT 0  
);  
  
CREATE TABLE weight_history (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  weight_kg REAL NOT NULL,  
  logged_at TEXT NOT NULL DEFAULT (datetime('now')),  
  note TEXT  
);
```

2.3 Reference Data & Recipes

```
CREATE TABLE usda_foods (  
  fdc_id INTEGER PRIMARY KEY,  
  description TEXT NOT NULL,  
  category TEXT,  
  calories_per_100g REAL NOT NULL,  
  protein_g_per_100g REAL NOT NULL,  
  carbs_g_per_100g REAL NOT NULL,  
  fat_g_per_100g REAL NOT NULL,  
  fiber_g_per_100g REAL  
);  
CREATE INDEX idx_usda_description ON usda_foods(description);  
CREATE VIRTUAL TABLE usda_foods_fts USING fts5(description, content='usda_foods',  
content_rowid='fdc_id');  
  
CREATE TABLE recipes (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  prep_time_minutes INTEGER,  
  servings INTEGER NOT NULL DEFAULT 1,  
  calories_per_serving REAL,  
  protein_g_per_serving REAL,  
  carbs_g_per_serving REAL,  
  fat_g_per_serving REAL,  
  instructions TEXT NOT NULL, -- JSON array of step strings  
  source TEXT NOT NULL DEFAULT 'generated' CHECK (source IN ('generated','saved','manual')),  
  created_at TEXT NOT NULL DEFAULT (datetime('now'))  
);  
  
CREATE TABLE recipe_ingredients (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  recipe_id INTEGER NOT NULL REFERENCES recipes(id) ON DELETE CASCADE,  
  name TEXT NOT NULL,  
  quantity TEXT NOT NULL  
);  
  
CREATE TABLE app_settings (  
  key TEXT PRIMARY KEY,  
  value TEXT NOT NULL  
);
```

03 — RUST LAYER

Backend Structure

```
src-tauri/
├── src/
│   ├── main.rs                // Tauri app bootstrap, plugin registration
│   ├── commands/
│   │   ├── mod.rs
│   │   ├── biometrics.rs      // BMR/TDEE/macro calculation commands
│   │   ├── logging.rs         // food_log CRUD commands
│   │   ├── vision.rs          // image encode + Ollama /api/chat orchestration
│   │   ├── recipes.rs         // pantry recipe generation commands
│   │   ├── lookup.rs          // USDA FTS5 search commands
│   │   └── export.rs          // CSV/JSON export commands
│   ├── ollama/
│   │   ├── client.rs          // reqwest-based REST client, localhost-only
│   │   └── schemas.rs         // serde structs mirroring the Zod contract
│   ├── db/
│   │   ├── mod.rs
│   │   ├── migrations.rs      // versioned, additive-only SQL migrations
│   │   └── models.rs          // typed row structs
│   └── calc/
│       └── nutrition.rs        // Mifflin-St Jeor, TDEE, macro-split pure functions
└── tauri.conf.json            // CSP: connect-src limited to http://localhost:11434
```

04 — INTERFACE

Tauri Command Surface

Command	Input	Returns
calculate_targets	profile + goal_type	Calorie & macro target set
save_profile	UserProfile	void
set_guardrails	diet[] + allergy[]	void
analyze_meal_photo	image path/bytes	MealAnalysis (validated)
save_food_log	FoodLogEntry	food_log_id
search_usda_foods	query string	FoodItem[] (ranked)
generate_recipes	pantry_items[] + guardrails	RecipeCandidate[]
get_progress_charts	date range	Weight/macro time-series
export_data	format ('csv' 'json'), dest path	file path written
check_ollama_status	—	{ running, models_installed[] }

05 — CONTRACT

Ollama Contract (Rust Side)

The Rust `ollama::schemas` module mirrors the frontend Zod schema exactly, so a payload that fails validation on one side is guaranteed to fail identically on the other.

```
// src-tauri/src/ollama/schemas.rs
use serde::{Serialize, Deserialize};

#[derive(Deserialize, Debug)]
pub struct FoodItem {
    pub name: String,
    pub estimated_grams: f32,
    pub calories: f32,
    pub protein_g: f32,
    pub carbs_g: f32,
    pub fat_g: f32,
    pub confidence: Confidence,
}

#[derive(Deserialize, Debug)]
#[serde(rename_all = "lowercase")]
pub enum Confidence { Low, Medium, High }

#[derive(Deserialize, Debug)]
pub struct MealAnalysis {
    pub items: Vec<FoodItem>,
    pub total_calories: f32,
}
```

06 — SPEED

Indexing & Performance

Table	Index	Why
usda_foods	idx_usda_description + FTS5 virtual table	Sub-5ms full-text search across ~400k reference rows.
food_log	idx_food_log_date on logged_at	Fast range queries for daily/weekly Dashboard rollups.
weight_history	idx_weight_date on logged_at	Efficient trend-line and 7-day-average queries.

07 — CHANGE MANAGEMENT

Migration Strategy

ADDITIVE-ONLY WITHIN A MINOR VERSION

Every migration adds columns/tables with safe defaults rather than dropping or renaming — this guarantees a user can downgrade one version without data loss. A versioned `schema_migrations` table tracks applied migrations, and a full `.db` file backup is copied automatically before any migration runs.